

LOGISTIC REGRESSION

Homework #3

Naomi Buell Richie Rivera Alexander Simon Nana Frimpong
Said Naqwe

2026-04-12

1 Introduction

We build a binary logistic regression model on a data set containing information on crime for various neighborhoods of a major city to predict whether the neighborhood will be at risk for high crime levels. Each record in our data has a response variable indicating whether or not the crime rate is above the median crime rate (1) or not (0). Our R statistical programming code is included in Appendix A.

2 Data Exploration

i Note

Completed

The crime training data set contains 466 rows and 13 features, and is 100 percent complete. The data are all numeric, and the target variable is **target**, which represents whether the crime rate is above the median crime rate (1) or not (0). The other variables represent various information on crime for neighborhoods of a major city. See Appendix B for a full list of variable names and definitions. Table 1 presents descriptive characteristics of the training data.

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness rate	Mean	Min	P25	P50	P75	Max
zn	1	12.000	0.00	0.00	0.00	16.00	100.00
indus	1	11.000	0.46	5.10	9.70	18.00	28.00
chas	1	0.071	0.00	0.00	0.00	0.00	1.00
nox	1	0.550	0.39	0.45	0.54	0.62	0.87
rm	1	6.300	3.90	5.90	6.20	6.60	8.80

Table 1: Summary statistics for all variables in the training dataset

Variable	Completeness rate	Mean	Min	P25	P50	P75	Max
age	1	68.000	2.90	44.00	77.00	94.00	100.00
dis	1	3.800	1.10	2.10	3.20	5.20	12.00
rad	1	9.500	1.00	4.00	5.00	24.00	24.00
tax	1	410.000	190.00	280.00	330.00	670.00	710.00
ptratio	1	18.000	13.00	17.00	19.00	20.00	22.00
lstat	1	13.000	1.70	7.00	11.00	17.00	38.00
medv	1	23.000	5.00	17.00	21.00	25.00	50.00
target	1	0.490	0.00	0.00	0.00	1.00	1.00

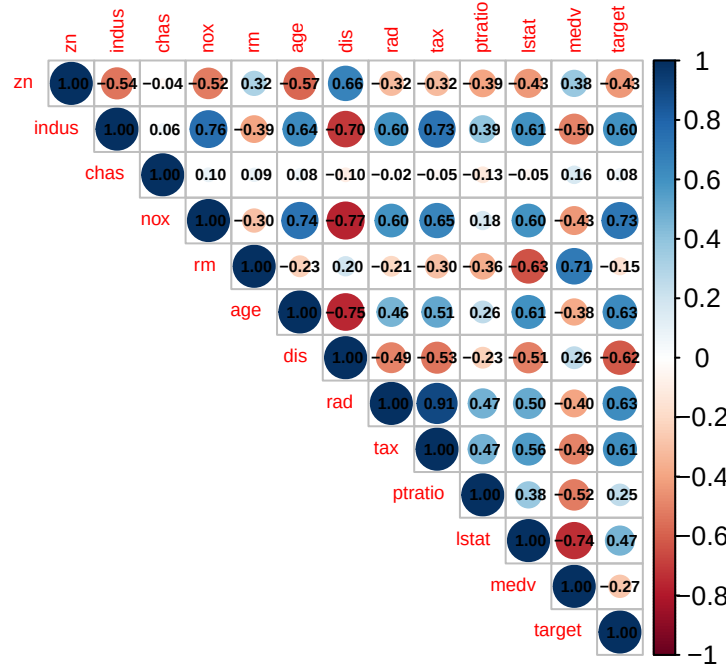


Figure 1: Correlation matrix of all variables in the training dataset

The correlation between all variables are shown Figure 1. The most correlated predictor variables with **target** are **nox** (nitrogen oxides concentration, $R = 0.73$), **age** (proportion of owner-occupied units built prior to 1940, $R = 0.63$), **rad** (index of accessibility to radial highways, $R = 0.63$), **dis** (weighted mean of distances to five Boston employment centers, $R = -0.62$), **tax** (full-value property-tax rate per \$10,000, $R = 0.61$), and **indus** (proportion of non-retail business acres per suburb, $R = 0.6$). These variables have the strongest linear relationships with the target variable and may be good predictors of crime rates.

Table 2: Multicollinear variables in the training dataset

Predictor 1	Predictor 2	Correlation
rad	tax	0.91
nox	dis	-0.77
indus	nox	0.76
age	dis	-0.75
lstat	medv	-0.74
nox	age	0.74
indus	tax	0.73
nox	target	0.73
rm	medv	0.71
indus	dis	-0.70

Table 2 shows predictor pairs with a pairwise correlation coefficient greater than 0.7. High pairwise correlation can create modeling issues because it becomes harder to isolate each predictor's individual effect on the target variable. The strongest multicollinear pair is **rad** (accessibility to highways) and **tax** (property-tax rate), which is plausible because neighborhoods with better highway access may also have higher property taxes. For this pair, and for the other pairs in the table, we should consider dropping one variable or using regularization techniques to mitigate multicollinearity.

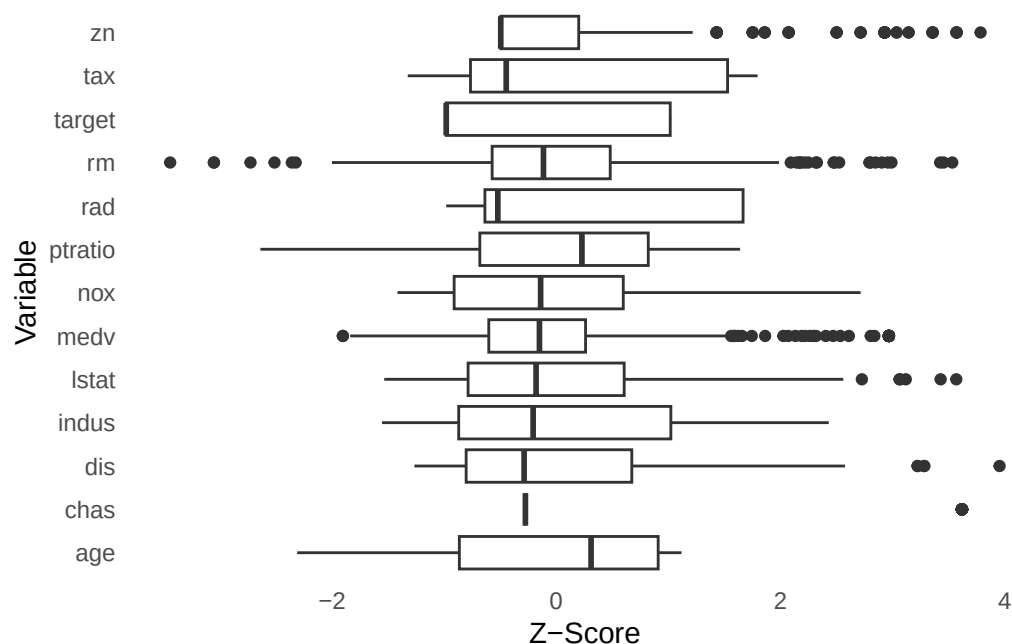


Figure 2: Box plots for identifying outliers in the training data

Figure 2 shows the distribution of z-scores for all variables and highlights potential outliers in the training data. The variable **zn** (proportion of residential land zoned for large lots over 25,000 square

feet) has many high outliers, which is plausible because some neighborhoods may have much higher shares of large lots than others; its maximum value is 100 percent. The variable `rm` (average number of rooms per dwelling) also has several outliers, ranging from 3.863 to 8.78 rooms. Likewise, `medv` (median value of owner-occupied homes) ranges from \$5k to \$50k, which is not unexpected given variation in housing prices across neighborhoods and unspecified data years. The variables `lstat` (percent lower status population) and `dis` (weighted mean distance to five Boston employment centers) show a few high outliers (37.97 percent and 12.1265 units, respectively). The variable `chas` has a high value of 1, which is expected because it is a binary indicator of whether a suburb borders the Charles River. Overall, these outliers appear to reflect real neighborhood variability rather than data-entry errors. If influence diagnostics later reveal problematic leverage points, we can consider adding indicator variables to account for them.

2.1 Data Preparation

Note

Starting from here, the following sections need to be updated for HW3 (currently carried over from HW1).

We handled high missingness in the variables `TEAM_BATTING_HBP`, `TEAM_BASERUN_CS`, and `TEAM_FIELDING_DP` (91.6%, 33.9%, and 12.6% missing, respectively), which are not very correlated with the target variable, by excluding these variables. The remaining variables had less than 10% missing, so we will impute the missing values with the median of each variable. We avoid dropping `TEAM_BASERUN_SB` as it is the most correlated variable with the target variable that has missingness, and instead impute it.

The median imputation was picked based on the distributions of missing variables: `TEAM_BASERUN_SB` and `TEAM_PITCHING_SO` have a long right tail so the median is a better imputation method than the mean. Both variables have observed have extreme outliers pulling the mean towards a right direction compared to where most the data sits. When the distribution of variables reveals two distinct peaks as observed in `TEAM_BATTING_SO`, we can refer to this distribution as bimodal. Since the mean of bimodal falls between two peaks, this area is observed to be the underrepresented region of the missing data. The median imputation is utilized to approach one of the two clusters of missing variables with the largest distribution of missing variables. Below are the histograms for the distributions of these variables.

```
# Create density curve for TEAM_BASERUN_SB
p1 <- ggplot(crime_train, aes(x = TEAM_BASERUN_SB)) +
  geom_density() +
  theme_minimal() +
  theme(panel.grid = element_blank(), axis.text = element_text(size = 7), axis.title = element_text(size = 7)) +
  xlab("TEAM_BASERUN_SB") +
```

```

    ylab("Density")

# Create density curve for TEAM_PITCHING_SO
p2 <- ggplot(crime_train, aes(x = TEAM_PITCHING_SO)) +
  geom_density() +
  theme_minimal() +
  theme(panel.grid = element_blank(), axis.text = element_text(size = 7), axis.title = element_text(size = 7))
  xlab("TEAM_PITCHING_SO") +
  ylab("")

# Create density curve for TEAM_BATTING_SO
p3 <- ggplot(crime_train, aes(x = TEAM_BATTING_SO)) +
  geom_density() +
  theme_minimal() +
  theme(panel.grid = element_blank(), axis.text = element_text(size = 7), axis.title = element_text(size = 7))
  xlab("TEAM_BATTING_SO") +
  ylab("")

# Combine the histograms
patchwork::wrap_plots(p1, p2, p3, ncol = 3) + patchwork::plot_layout(guides = 'collect')

```

To begin by remove variables with high missingness variables and low correlation with **TARGET_WINS**: **TEAM_BATTING_HBP** (91.6% missing, $r=0.07$), **TEAM_BASERUN_CS** (33.9% missing, $r=0.02$), and **TEAM_FIELDING_DP** (12.6% missing, $r=-0.03$).

```

# Drop high missingness variables and low-correlation variables with the data
# These variables lack a strong relationship with response variable TARGET_WINS
crime_clean <- crime_train |>
  select(-TEAM_BATTING_HBP, -TEAM_BASERUN_CS, -TEAM_FIELDING_DP)

crime_eval_clean <- crime_eval |>
  select(-TEAM_BATTING_HBP, -TEAM_BASERUN_CS, -TEAM_FIELDING_DP)

```

Before our predictor variables **TEAM_BASERUN_CS**, **TEAM_BASERUN_SB** and **TEAM_PITCHING_SO** are imputed, a binary indicator is created to preserve information that the value was originally missing. Our indicator for each flag is (1 = missing, 0 = present).

```

crime_clean <- crime_clean |>
  mutate(
    TEAM_BASERUN_SB_MISSING = as.integer(is.na(TEAM_BASERUN_SB)),
    TEAM_BATTING_SO_MISSING = as.integer(is.na(TEAM_BATTING_SO)),

```

```

    TEAM_PITCHING_SO_MISSING = as.integer(is.na(Team_Pitching_SO))
  )

crime_eval_clean <- crime_eval_clean |>
  mutate(
    TEAM_BASERUN_SB_MISSING = as.integer(is.na(Team_Baserun_SB)),
    TEAM_BATTING_SO_MISSING = as.integer(is.na(Team_Batting_SO)),
    TEAM_PITCHING_SO_MISSING = as.integer(is.na(Team_Pitching_SO))
  )

```

Imputation is now based on the distribution of missing data of the following variables. TEAM_BASERUN_SB, TEAM_PITCHING_SO, and TEAM_BATTING_SO. TEAM_BASERUN_SB and TEAM_PITCHING_SO are right skewed, therefore they are imputed around the mean. TEAM_BATTING_SO is bimodal, so we can impute around the median to avoid placing the imputed values in region between two peaks where only a few observations exist.

```

crime_clean <- crime_clean %>%
  mutate(
    # Mean imputation for right-tailed variables
    TEAM_BASERUN_SB = ifelse(
      is.na(Team_Baserun_SB),
      mean(Team_Baserun_SB, na.rm = TRUE),
      Team_Baserun_SB
    ),
    TEAM_PITCHING_SO = ifelse(
      is.na(Team_Pitching_SO),
      mean(Team_Pitching_SO, na.rm = TRUE),
      Team_Pitching_SO
    ),

    # Median imputation for bimodal variable
    TEAM_BATTING_SO = ifelse(
      is.na(Team_Batting_SO),
      median(Team_Batting_SO, na.rm = TRUE),
      Team_Batting_SO
    )
  )

crime_eval_clean <- crime_eval_clean %>%
  mutate(

```

```

# Mean imputation for right-tailed variables
TEAM_BASERUN_SB = ifelse(
  is.na(TEAM_BASERUN_SB),
  mean(TEAM_BASERUN_SB, na.rm = TRUE),
  TEAM_BASERUN_SB
),
TEAM_PITCHING_SO = ifelse(
  is.na(TEAM_PITCHING_SO),
  mean(TEAM_PITCHING_SO, na.rm = TRUE),
  TEAM_PITCHING_SO
),

# Median imputation for bimodal variable
TEAM_BATTING_SO = ifelse(
  is.na(TEAM_BATTING_SO),
  median(TEAM_BATTING_SO, na.rm = TRUE),
  TEAM_BATTING_SO
)
)

```

When a distribution is observed to be skewed to the right, meaning small number of outliers has influence on the response variable `TARGET_WINS`, the log transformation can be applied to balance the scale of distribution. Utilizing log compresses the distribution of missing variable closer to symmetric shape. This makes the relationship with `TARGET_WINS` more linear and easier for the model to capture. `log1p` is used to handle any zero without producing undefined results.

```

# Log transform right-skewed variables
crime_clean <- crime_clean %>%
  mutate(
    LOG_TEAM_BASERUN_SB = log1p(TEAM_BASERUN_SB),
    LOG_TEAM_PITCHING_SO = log1p(TEAM_PITCHING_SO),
    LOG_TEAM_FIELDING_E = log1p(TEAM_FIELDING_E)
  )

crime_eval_clean <- crime_eval_clean %>%
  mutate(
    LOG_TEAM_BASERUN_SB = log1p(TEAM_BASERUN_SB),
    LOG_TEAM_PITCHING_SO = log1p(TEAM_PITCHING_SO),
    LOG_TEAM_FIELDING_E = log1p(TEAM_FIELDING_E)
  )

```

We can create three new variables that captures negative and positive impact on the wins for the team.

```
# HR_DIFFERENTIAL and BAT_BB_SO_RATIO
crime_clean <- crime_clean |>
  mutate(
# Net home run differential (offensive power vs pitching vulnerability)
# The team that hits more home runs than it gives up will produce positive value
# the team that allows more home runs than it hits will produce a negative
#value, meaning they losing the game
# Zero indicates the team is even in home runs
    HR_DIFFERENTIAL = TEAM_BATTING_HR - TEAM_PITCHING_HR,
# Here we can measure how batter can judge which pitch they would take a swing at
# If the ratio is greater than 1, it reveals the team walk more than they strike
# showing a more cohesive and patient team
# This may happen if a batter force the pitchers to throw more pitches

    BAT_BB_SO_RATIO = ifelse(TEAM_BATTING_SO > 0,
                              TEAM_BATTING_BB / TEAM_BATTING_SO, NA)
  )

crime_eval_clean <- crime_eval_clean |>
  mutate(
    HR_DIFFERENTIAL = TEAM_BATTING_HR - TEAM_PITCHING_HR,
    BAT_BB_SO_RATIO = ifelse(TEAM_BATTING_SO > 0,
                              TEAM_BATTING_BB / TEAM_BATTING_SO, NA)
  )

# Impute missing values in BAT_BB_SO_RATIO with median
crime_clean <- crime_clean |>
  mutate(
    BAT_BB_SO_RATIO = ifelse(is.na(BAT_BB_SO_RATIO),
                              median(BAT_BB_SO_RATIO, na.rm = TRUE), BAT_BB_SO_RATIO)
  )

crime_eval_clean <- crime_eval_clean |>
  mutate(
    BAT_BB_SO_RATIO = ifelse(is.na(BAT_BB_SO_RATIO),
                              median(BAT_BB_SO_RATIO, na.rm = TRUE), BAT_BB_SO_RATIO)
  )
```


Table 3

```
# Verify that no variables have remaining missingness
crime_clean |>
  summarise(across(everything(), ~ sum(is.na(.)))) |>
  tidyr::pivot_longer(everything(),
                      names_to = "Variable",
                      values_to = "Missing") |>
  filter(Missing > 0) |>
  nrow()
```

Finally we can verify that there no missing variables that remain based on the results of our imputation.

2.2 Verifying Linearity Assumption

Linear regression makes four important assumptions:

1. The independent and dependent variables are linearly related.
2. The errors are independent of each other.
3. The errors have constant variance.
4. The errors are normally distributed.

The last three will be evaluated after models are built (section 3). Here, we test the assumption of linearity using pair plots of the variables in the cleaned training dataset. To avoid overcrowded plots, we divided the variables into three groups.

2.2.1 Batting and baserun variables

- In the first row of Figure 1, the correlation coefficients show that all batting and baserun variables except TEAM_BATTING_SO have significant correlations with the target variable.
- In the first column, the scatterplots show that TEAM_BATTING_H and TEAM_BATTING_2B have the clearest linear relationship with TARGET_WINS.
- There are no obvious nonlinear relationships between batting and baserun variables and the target variable.

```
# This block takes a few seconds
crime_clean |>
  select(-c(ends_with('MISSING'))) |> # omit categorical (flag) variables
  select(TARGET_WINS, starts_with('TEAM_BATTING'), TEAM_BASERUN_SB) |>
  ggpairs(
    # Color transparency and small data points help reduce overplotting
    mapping = aes(alpha = 0.1),
```

```

lower = list(continuous = wrap('points', size = 0.01)),
progress = FALSE
) +
theme(
  strip.text = element_text(size = 8)
)

```

2.2.2 Pitching and fielding variables

- In the first row, the correlation coefficients show that all pitching variables and the fielding variable have significant correlations with the target variable.
- In the first column, the scatterplots show that `TEAM_PITCHING_BB` has the clearest linear relationship with `TARGET_WINS`.
 - Two independent variables (`TEAM_PITCHING_H` and `TEAM_PITCHING_SO`) appear to be related by a horizontal line. Technically, this satisfies the assumption of linearity, but it suggests that these variables do not have predictive value.
- There are no obvious nonlinear relationships between pitching and fielding variables and the target variable.

```

# This block takes a few seconds
crime_clean |>
  select(-c(ends_with('MISSING')))) |> # omit categorical (flag) variables
  select(TARGET_WINS, starts_with('TEAM_PITCHING'), TEAM_FIELDING_E) |>
  ggpairs(
    mapping = aes(alpha = 0.1),
    lower = list(continuous = wrap('points', size = 0.01)),
    progress = FALSE
  ) +
  theme(
    strip.text = element_text(size = 8)
  )

```

2.2.3 New variables

- In the first row, the correlation coefficients show that four of the five newly created variables have significant correlations with the target variable. The exception was `LOG_TEAM_PITCHING_SO`, which had a correlation coefficient of 0.003.
- In the first column, the scatterplots show that `BAT_BB_SO_RATIO` has the clearest linear relationship with `TARGET_WINS`.

- LOG_TEAM_PITCHING_SO appears to be related by a horizontal line. Although this satisfies the assumption of linearity, it suggests that this variable may not have predictive value.
- There are no obvious nonlinear relationships between the newly created variables and the target variable.

```
# This block takes a few seconds
# Pair plots for new variables
crime_clean |>
  select(TARGET_WINS, starts_with('LOG'), HR_DIFFERENTIAL, BAT_BB_SO_RATIO) |>
  ggpairs(
    mapping = aes(alpha = 0.1),
    lower = list(continuous = wrap('points', size = 0.01)),
    progress = FALSE
  ) +
  theme(
    strip.text = element_text(size = 8)
  )
```

2.2.4 Linearity Assumption Summary

The results above indicate that the assumption of linearity is satisfied for all independent variables. However, several were flagged as being unlikely to have predictive value in a multivariate linear model.

2.3 Build Models

First, we will split our dataset into a training and testing set. We will use the training set to build our models and the testing set to evaluate their performance. The training set will be used to fit the models, while the testing set will be used to assess how well the models generalize to new data.

```
set.seed(19940211)
train_indices <- sample(nrow(crime_clean), size = 0.8 * nrow(crime_clean))
crime_train_split <- crime_clean[train_indices, ]
crime_test_split <- crime_clean[-train_indices, ]
```

We will start by building three models. The first model contains all the variables in the cleaned training dataset. The second model only includes the variables that are most correlated with the target variable and the two new predictors (HR_DIFFERENTIAL and BAT_BB_SO_RATIO). Finally, we'll create a third model using automated stepwise selection to identify the best subset of predictors. This method iteratively adds or removes predictors to find the model that best balances fit and complexity.

We will compare the performance of these two models to determine whether including all variables improves performance compared to a model with just the most correlated variables. The stepwise selection model will help us identify whether there are any additional predictors that improve performance beyond the most correlated variables and the new predictors. We will evaluate the performance of these models in the next section to determine which one is best for predicting `TARGET_WINS`.

```
# First model with all variables
model_full <- lm(
  TARGET_WINS ~ .,
  data = crime_train_split |>
    select(-INDEX)
)

# Second model with most correlated variables and new predictors
model_theory <- lm(
  TARGET_WINS ~ TEAM_BATTING_H + TEAM_BATTING_2B + TEAM_BATTING_BB + HR_DIFFERENTIAL + BAT_BB_9
  data = crime_train_split
)

# A third model using Automated Stepwise Selection
model_stepwise <- step(
  model_full,
  direction = "backward",
  trace = 0
)
```

2.3.1 Verifying Remaining Assumptions for Linear Regression

Prior to comparing the models, we will check the remaining assumptions of linear regression. Restating the assumptions:

2.3.1.1 The independent and dependent variables are linearly related.

This was verified above during the data exploration phase using pair plots. The scatterplots in the pair plots showed that there were no obvious nonlinear relationships between the independent variables and the target variable `TARGET_WINS`. Therefore, the assumption of linearity is satisfied.

2.3.1.2 The errors are independent of each other.

To check for independence of errors, we can plot the auto-correlations plots (ACF) of the residuals for each model. If there are significant autocorrelations at any lag, this suggests that the errors are

not independent.

```
par(mfrow = c(1, 3))
acf(residuals(model_full), main="ACF: Full Model")
acf(residuals(model_theory), main="ACF: Theory Model")
acf(residuals(model_stepwise), main="ACF: Stepwise Model")
```

These ACF plots show that there are just a few significant autocorrelations from lag 1 to lag 10 for all three models, which suggests that the errors are not independent. This may be due to the fact that the data is time series data, and there may be temporal dependencies in the errors. This would make sense as the performance of a baseball team in one year may be related to its performance in the previous year and this data is defined as “[representing] a professional baseball team from the years 1871 to 2006 inclusive. Each record has the performance of the team for the given year, with all of the statistics adjusted to match the performance of a 162 game season.”

2.3.1.3 The errors have constant variance (Homoscedasticity).

To check for constant variance, we examine the Residuals vs Fitted plots and the Scale-Location plots for our models. We are looking for a relatively random scatter of points around the horizontal zero line. If the spread of the residuals increases or decreases systematically with the fitted values (forming a “megaphone” or “funnel” shape), it indicates heteroscedasticity.

```
# Set up a 2x3 grid for the plots
par(mfrow = c(2, 3))

# Row 1: Residuals vs Fitted
plot(model_full, which = 1, main = "Resid vs Fitted (Full)")
plot(model_theory, which = 1, main = "Resid vs Fitted (Theory)")
plot(model_stepwise, which = 1, main = "Resid vs Fitted (Step)")

# Row 2: Scale-Location
plot(model_full, which = 3, main = "Scale-Location (Full)")
plot(model_theory, which = 3, main = "Scale-Location (Theory)")
plot(model_stepwise, which = 3, main = "Scale-Location (Step)")
```

The residuals vs fitted plots across all three models show that the residuals are randomly scattered around the horizontal zero line, which is evidence that the model adheres to the assumption of homoscedasticity. The scale-location plots show a slightly different story where there's a consistent ‘U’ or ‘V’ shape with the red line indicating that the variance of the residuals is not constant across the range of fitted values. This suggests that there may be some heteroscedasticity in the models, which could affect the reliability of the standard errors and confidence intervals.

2.3.1.4 The errors are normally distributed.

To verify that our residuals follow a normal distribution, we use Normal Q-Q plots. If the assumption holds then the standardized residuals should be in a diagonal line. Deviations at the lower or upper tails suggest skewed or heavy-tailed errors. We will also perform a Shapiro-Wilk statistical test to quantitatively assess normality, where a p-value < 0.05 indicates a significant departure from a normal distribution.

```
# Set up a 1x3 grid for the plots
par(mfrow = c(1, 3))
plot(model_full, which = 2, main = "Normal Q-Q (Full)")
plot(model_theory, which = 2, main = "Normal Q-Q (Theory)")
plot(model_stepwise, which = 2, main = "Normal Q-Q (Step)")

# Formal Shapiro-Wilk Test (Note: p-value < 0.05 means NOT normal)
shapiro.test(residuals(model_full))
shapiro.test(residuals(model_theory))
shapiro.test(residuals(model_stepwise))
```

For all 3 models the points follow the diagonal line reasonably well in the middle of the distribution but the upper tail curves upwards and the lower tail curves downwards. This suggests that the residuals have heavier tails than a normal distribution, which may indicate the presence of outliers or a non-normal error distribution.

From the Shapiro-Wilk tests, we find that the p-values for all three models are much less than 0.05 which indicates that the residuals significantly deviate from a normal distribution. This is consistent with the observations from the Q-Q plots and suggests that the assumption of normality is violated for all three models. This violation is likely due to the presence of outliers or a heavy-tailed error distribution and we can likely continue with the models but we should be cautious when interpreting the results.

2.4 Select Models

```
# Creating a custom helper function to extract all the required metrics
# explicitly evaluating on the provided training dataset.
get_model_metrics <- function(model, model_name, train_data) {
  mod_summary <- summary(model)

  # Explicitly calculate Mean Squared Error (MSE) using the training data
  predictions <- predict(model, newdata = train_data)
  mse <- mean((train_data$TARGET_WINS - predictions)^2)
```

```

# Extract F-statistic and calculate its p-value (inherent to training data)
f_stat <- mod_summary$fstatistic
p_val <- if (!is.null(f_stat)) {
  pf(f_stat[1], f_stat[2], f_stat[3], lower.tail = FALSE)
} else {
  NA
}

# Return a nicely formatted row for our table
tibble(
  Model = model_name,
  `Training MSE` = mse,
  `R-Squared` = mod_summary$r.squared,
  `Adj R-Squared` = mod_summary$adj.r.squared,
  `F-Statistic` = f_stat[1],
  `P-Value` = p_val,
  `AIC` = AIC(model),
  `BIC` = BIC(model)
)
}

# Apply the function to all three models, explicitly passing the training split
model_comparison <- bind_rows(
  get_model_metrics(model_full, "Full Model", crime_train_split),
  get_model_metrics(model_theory, "Theory Model", crime_train_split),
  get_model_metrics(model_stepwise, "Stepwise Model", crime_train_split)
) |>
# Sort by the lowest Training MSE
arrange(`Training MSE`)

# Display the table
knitr::kable(
  model_comparison,
  digits = 4,
  caption = "Evaluation of Candidate Models on Training Dataset"
)

```

We favor a slightly more parsimonious model over a complex one with a marginally better R^2 . A simpler model (like our Stepwise or Theory model) is easier to interpret and less prone to overfitting.

We will evaluate the models using Adjusted R^2 and MSE on the training data, and then use the evaluation dataset to compare their predictive performance.

Looking at the table above, we can see that the Stepwise and Full models have similar MSE and Adjusted R^2 , but the Stepwise model has fewer predictors making it a bit more parsimonious. Additionally, the Stepwise model is more interpretable and less prone to overfitting than the Full model. Additionally, the theory model has a much higher MSE and lower Adjusted R^2 than the other two models, which suggests that baseball is more complex to predict wins with just the most correlated variables and the two new predictors. Therefore, **we select the Stepwise model as our final model for predicting TARGET_WINS.**

Regarding the F-statistic, the highly significant p-value for the stepwise model's F-statistic (119.9) confirms that our selected group of predictors is collectively highly effective at explaining the variance in total team wins.

```
# Display the summary of the selected Stepwise model to interpret coefficients and significance
summary(model_stepwise)
```

To make inferences from our Stepwise model, we can interpret the coefficients. For example, the coefficient for `TEAM_BATTING_H` is 0.05 which indicates that, all other things being equal, for every additional 20 hits a team records over a season, they're expected to gain about 1 additional win. Similarly, `TEAM_BATTING_BB` (i.e., walks) has a positive highly significant coefficient (0.03) which aligns with baseball theory that teams that walk more tend to win more.

3 Results

```
# Generate predictions on the evaluation dataset using the selected model
eval_predictions <- predict(model_stepwise, newdata = crime_eval_clean)

# Bind the predictions back to the evaluation dataset for easy viewing
crime_eval_results <- crime_eval_clean |>
  # We round the predictions to the nearest whole number because a team
  # can't win a fraction of a baseball game
  mutate(PREDICTED_TARGET_WINS = round(eval_predictions, 0)) |>
  # Move the new prediction column to the front (right after INDEX) for easy reading
  relocate(PREDICTED_TARGET_WINS, .after = INDEX)

# Write predictions to a csv
write_csv(crime_eval_results, "crime_evaluation_predictions.csv")

# Display the first 10 predictions in the rendered PDF
```



```
knitr::kable(
  head(crime_eval_results |> select(INDEX, PREDICTED_TARGET_WINS), 10),
  caption = "Excerpts of Stepwise Model Win Predictions for Evaluation Set"
)
```

Given our evaluation set and our stepwise model, we are able to create predictions for the number of wins for the team in each year. The first 10 predictions are shown in the table above. We write these predictions to a CSV file for submission.

4 Conclusion

In conclusion, our stepwise model performed best in predicting the number of baseball wins based on the training data. Since there were some minor violations of assumptions, these results should be interpreted with caution, which may affect the reliability of our model. Future work could explore data transformations or alternative modeling approaches to address these assumption violations and potentially improve predictive performance.

5 Appendix A: R Code

Note

If we want to include our R statistical programming code in an Appendix, add all code here and set the chunk option `include = false` to hide the code from the main report.

6 Appendix B: Data Dictionary

Variable	Description	Role
zn	Proportion of residential land zoned for large lots (over 25,000 square feet)	Predictor
indus	Proportion of non-retail business acres per suburb	Predictor
chas	Dummy variable for whether the suburb borders the Charles River (1) or not (0)	Predictor
nox	Nitrogen oxides concentration (parts per 10 million)	Predictor
rm	Average number of rooms per dwelling	Predictor

Variable	Description	Role
age	Proportion of owner-occupied units built prior to 1940	Predictor
dis	Weighted mean of distances to five Boston employment centers	Predictor
rad	Index of accessibility to radial highways	Predictor
tax	Full-value property-tax rate per \$10,000	Predictor
prratio	Pupil-teacher ratio by town	Predictor
lstat	Lower status of the population (percent)	Predictor
medv	Median value of owner-occupied homes in \$1000s	Predictor
target	Whether the crime rate is above the median crime rate (1) or not (0)	Response